From Operation Logs to an Automation Proposal

Final report · Repository: https://github.com/insaneado/iby

Summary

Dataset B contains **175 minutes of back-office work: 668 executions across 15 sessions and 4 operators**, all on 2026-07-01. Recovering that required solving the problem the raw logs pose — they record keystrokes and clicks, but nothing that says which unit of work is in progress.

The three findings that drove everything else:

1. **The case identifier is recoverable from the screen.** 97.8% of ground-truth case IDs appear in captured screen text. This turns Step 1 from blind change-point detection into case-identity reconstruction — the only method that can separate two consecutive executions of the *same* process.
2. **Every unit of work ends with a button press.** On dataset A those 1,751 presses land inside a gold segment 1,751 times out of 1,751, exactly one per segment, at median relative position 0.89. Making the press define the segment end lifted boundary F1 from 0.450 to **0.722**.
3. **One screen pattern carries 35.7% of all observed work**, in all three portal systems, with an identical table contract. That is what the automation targets, and why it is one engine with three definitions rather than three scripts.

The delivered tool processes **456 worklist rows with 94% fully automated and zero failures**. The 6% left for a person are precisely the rows where the governing regulation states no applicable rule.

The recommendation I would defend hardest is a negative one: **the LLM does not belong in the runtime path**, and the evidence for that is in §5.

1. Step 1 — recovering units of work

What the logs do and do not contain

Process mining needs an event log of *(case ID, activity, timestamp)*. The provided data has timestamps only. Recovering the other two from an uncased stream is the known problem of *event log correlation*.

What did not work, and why I checked first

The obvious approach is to segment on pauses. I measured the gap distribution before building it: **the median gap between consecutive ground-truth segments is 0.0 seconds** — half of all true boundaries have no pause at all. Built anyway as a baseline, it reached BF1@5s 0.314 and V-measure 0.063. There is no threshold at which it works: at 2 s it emits 8,290 segments for 2,009 real ones; at 10 s it misses 84% of boundaries.

A second baseline, splitting on every application switch, was also poor (V = 0.135): operators switch applications constantly *within* one unit of work.

The approach that did work

stage	signal	why
case identity	most-repeated ID in a screen capture	opening a record repeats its ID across header, fields and breadcrumb; list rows show each once. 93.5% precision
boundary	click on an HTML `button`	the terminal action of a unit; 1,751/1,751 inside a gold segment, one per segment
label	portal system + route	3 systems x 5 routes is exactly the 15 process families; V = 0.854 on gold segments

Results

Scored against dataset A's 2,009 ground-truth executions:

	best baseline	delivered (v3)	ground truth
boundary F1 @2s	0.226	0.286	1.000
boundary F1 @5s	0.314	**0.722**	1.000
boundary F1 @10s	0.494	0.765	1.000
WindowDiff (lower better)	0.656	0.294	0.000
V-measure (label consistency)	0.135	0.518	1.000
segments produced	4,150	**2,015**	2,009
idle time claimed	41.5%	**5.7%**	5.0%

Two figures were never optimised for and are the ones I trust most: the segment count lands within 0.3% of ground truth, and claimed idle time within 0.7 points.

How honest this number is

- **The scorer was validated first.** Scoring ground truth against itself returns 1.000 on every metric. Without that check the rest is unfalsifiable.
- **Two audits found real defects.** The boundary metric originally derived boundaries from label changes, which made the boundary between two consecutive executions of the *same* process invisible — it was scoring 94% of boundaries and silently discarding exactly the hardest 6%. A second pass found the fix had reached boundary F1 but not WindowDiff.
- **Parameter tuning leaked.** Parameters were swept over all 63 sessions with a dev/test split reported afterwards. Redone strictly, the honest protocol scores *higher* on test (0.745 vs 0.722) — the leak was real and was not inflating the result.
- **Generalisation was tested by holding out whole operators**, not random sessions: **BF1@5s 0.728, sd 0.120** across seven held-out machines.

Where it fails, precisely

One held-out machine scores **0.444** against 0.72–0.82 for the other six. It is not a tuning artefact: **LAPTOP-R36BQBTE has zero L3 events across all seven of its sessions** — the browser extension never connected, so the

route signal does not exist there. A fallback that recovers the route from the L2 accessibility layer limits the damage but does not remove it.

That gives the single most useful number in this report for planning purposes: **expect BF1@5s ≈ 0.73 ± 0.12 on an unseen operator, falling to ≈ 0.44 wherever L3 capture is missing.**

Boundary precision at 2-second tolerance is also weak (0.286) and structurally so: screen text is captured every ~7.7 s, so no anchor can localise a change more finely. Improving it would mean inventing boundaries between observations.

Deliverable: `out/segments.jsonl` — 668 segments, 15 sessions, 14 labels.

---

2. Step 2 — what the work is

Scale and people

668 executions, 175 minutes, 15 sessions, 4 operators.

Three sources disagreed on headcount and had to be reconciled: 4 `username_hash` values, 4 machine IDs (strictly 1:1), but only 3 operator names on screen. The names are **not** operators — all three appear in every session, and each maps to one portal system at 97–100% consistency. They are **shared per-system logins**. So: 4 people, working across three systems under shared accounts. That last part is a governance finding, not trivia (§6).

The route names are misleading

The portal is one SPA deployed three times, so route names repeat and describe nothing. The regulation document open during the work identifies it:

segment label	document consulted	what the work is
`ops__leave-applications`	keiyaku_kaijo_tetsuzuki	contract termination
`fin__leave-applications`	settai_keihi_kitei	entertainment expense approval
`fin__payroll-items`	getsujitsu_teigaku_torihikisaki_ichiran	recurring supplier payment
`fin__resident-tax`	shinkuitorihikisaki_touroku_tetsuzuki	new supplier registration
`hr__onboarding`	nyusha_checklist_shinsotsu_batch	new-graduate onboarding
`ops__social-insurance`	kanrisya_kengen_shinsei_tetsuzuki	admin privilege request

This doubles as **independent validation of Step 1's labels**: the pipeline never reads document names, yet segments sharing a label consult the same document at **79% weighted purity**.

(All Japanese readings in this report were verified by a Japanese-reading reviewer on 2026-09-10 — see `docs/CHECK_THIS.md`.)

Ranking, and why it is ranked this way

Two measurable quantities in tension:

- **Mechanical load** — clipboard events and application switches per run. What automation removes.
- **Judgment load** — share of runs with a regulation document open. What it does not.

High volume with high judgment is a poor first target however expensive it looks, because the residue is what costs the time.

process	n	min	share	median	mech	judgment
payroll_item_maintenance	115	24.5	14.0%	11 s	2.3	11%
recurring_supplier_payment	74	22.8	13.1%	15 s	3.5	55%
contract_termination	65	22.0	12.6%	18 s	6.3	79%
new_grad_onboarding	59	20.2	11.5%	19 s	6.3	83%
inventory_payroll_items	65	15.1	8.6%	11 s	3.7	6%
new_supplier_registration	67	13.8	7.9%	11 s	1.9	22%

A priority formula is easy to make say what you want, so the ranking was scored under **six different weightings**. Only **payroll_item_maintenance** appears in the top three under all six. Four processes appear exactly once, which makes them artefacts of a particular formula rather than findings.

The result that set the scope

Aggregating by portal **screen** instead of by system:

screen	executions	minutes	share	systems	judgment
payroll-items	**254**	**62.4**	**35.7%**	**3**	**24%**
leave-applications	159	44.6	25.5%	3	48%
onboarding	99	32.2	18.4%	2	72%
social-insurance	86	20.4	11.7%	3	23%
resident-tax	67	13.8	7.9%	1	22%

The three top-ranked processes are the same screen in three deployments. One pattern, 36% of all work, second-lowest judgment load. That is the target.

---

3. Step 3 — what I built

A **shared worklist automation engine** driven by a per-system definition file, implemented for all three systems.

```
 tool/engine.py              the automation logic - one copy
tool/definitions/*.yaml     what differs per system: URL, regulation, wording
tool/regulations.py         extracts decision rules from the 規程 text
tool/mock_portal/           the portal, reconstructed from the logs
```

Measured result — all 456 worklist rows

outcome	rows	share
routine (定常), templated note	336	74%
adjustment (調整), routed by regulation threshold	94	21%
fully automated	**430**	**94%**
left for a person	26	6%
failed	**0**	**0%**

Median 151 ms per row, p95 224 ms, 89 s for the full set.

Why this process and this scope

Why this process: it is the only candidate that survived all six ranking weightings, and its screen pattern accounts for 35.7% of observed work.

Why this scope: the brief notes that breadth-versus-depth is itself the ROI question. The deciding evidence is that all three systems share an *identical table contract* — same seven columns, same element ids, differing only in content (`E2001` vs `BATCH-W2`, yen vs an em dash). Three bespoke scripts would encode that structure three times and triple the maintenance for no extra coverage. One engine plus three ~30-line definitions covers 36% of the work.

What I deferred: the other four screens. `leave-applications` is the next largest at 25.5% but carries double the judgment load, and `onboarding` (72% judgment) is largely procedural checklist work that the threshold-rule approach does not address. Extending to those means solving procedure-following, which is a different and harder problem.

---

4. Why this implementation form, and what I rejected

option	why not
RPA tool (UiPath, Power Automate)	Would work. Rejected on operational grounds: the client already captures DOM selectors, so a code path is more testable, diffable and reviewable than a recorded flow — and the per-system difference is data, which suits a config file rather than three recorded macros.
An LLM agent driving the UI	Measured and rejected. See below.
API integration	Preferable if it exists, but nothing in the logs evidences an API. Proposing one would be an assumption, and the brief warns against exactly that. Flagged as the first thing to check in a real engagement (§6).
Three separate scripts	Rejected on the identical-table-contract evidence above.
Deterministic engine + config	**Chosen.**

The LLM decision, in detail

This was designed as a RAG feature: read the 規程, decide the handling. Two pieces of evidence removed it.

Measured performance. Running the judgment step through Gemini:

	deterministic	model
median latency per row	170 ms	23,310 ms (**135x**)
errors	0 / 456	2 / 5 (timeouts)
output quality	states the facts	echoed the regulation's *filename* back

And then the reason it was never needed. The captured regulation text is a threshold table, not a judgment:

> 第 2 条（承認権限）1回あたり5万円未満：部門長承認。5万円以上：役員承認。10万円以上：社長承認。
>
> *Article 2 (Approval authority). Under ¥50,000 → department head; ¥50,000 and over → executive; ¥100,000 and over → president.*

Approval routing by amount is **arithmetic**. Arithmetic belongs in code: exact, instant, free, auditable line by line against the regulation, and incapable of inventing an approver. `regulations.py` parses those thresholds; `route(107158)` returns 社長承認; the tool writes `規程により社長承認へ回付`.

The model keeps exactly one job, and it runs offline: proposing a rule table from a regulation document for a human to check before it ships. The expensive, unreliable, unauditable component runs *once under supervision* rather than on every transaction. The tool runs identically with no model configured.

I would defend this as the correct answer rather than a compromise. An LLM in this path would have been slower, less reliable, more expensive, unauditable against the regulation it is supposed to implement — and would have looked more impressive.

---

5. What manual work remains, and realistic impact

Remains, by construction

- **26 of 456 rows (6%)** — inventory adjustments whose 金額 is an em dash. With no amount, the threshold table says nothing. Correct boundary, not a gap.
- **The other 10 of 13 regulation documents** carry no machine-readable thresholds; they are procedural checklists. Processes governed by those are out of scope entirely.
- **Exception handling.** Nothing in the logs shows what operators do when a record is malformed or a system rejects a submission, so the tool has no designed behaviour for it beyond failing loudly.
- **Review of the automated output.** At least during rollout, a person should sample what the tool wrote.

Impact, stated carefully

The brief states the recordings were made in a test environment with compressed waiting times, so **absolute durations here cannot be converted into hours saved, and I will not do so.** What the evidence supports is

relative:

- The target screen pattern is **35.7% of observed work**.
- Within it, **94% of rows** were handled end to end without a person.
- So the defensible claim is that the tool addresses roughly **a third of observed back-office time, at 94% coverage within that third** — under favourable conditions (§6).

What it does *not* support: any statement of hours or money saved. Producing one would require production timings the client has and I do not.

---

6. Risks, and the evidence for each

Every risk below is anchored to a measurement rather than to a worry.

#	risk	evidence	mitigation
R1	**Telemetry gaps silently degrade accuracy.**	One of seven held-out machines scored BF1@5s **0.444** vs 0.72–0.82. Cause: **zero L3 events across all 7 of its sessions** — the extension never connected.	Monitor L3 coverage per machine as a first-class health metric; refuse to report process figures for a machine below a coverage floor. The L2 accessibility fallback limits but does not remove the damage.
R2	**The mock portal is not the real portal.**	Session handling, server-side validation, pagination, concurrency and real latency are **unobserved in the logs** and therefore unimplemented.	Treat 94% as an upper bound under favourable conditions. First engagement task: run against a staging instance before any efficiency claim is repeated.
R3	**An approach validated on one department can fail silently on another.**	Three transfers failed during this project: the case-ID prefix (100% pure on A, meaningless on B), the anchor rule (fired 72 times in all of B), and the button naming (`btn-*-ok` matched **zero** rows in A). Each looked fine on internal statistics.	Never accept a transfer on internal statistics alone. Hold back one observable signal from the method and check against it — that is exactly what caught the first dataset B failure.
R4	**Automation runs under a shared account.**	Each portal system has one login shared by all four operators (97–100% name-to-system consistency).	No per-user audit trail exists today, so automated and human actions will be indistinguishable in the client's own logs. Needs a service account with a distinct identity before rollout, which is an access-control change, not a code change.
R5	**The regulation is the specification, and it changes.**	Thresholds are hard rules parsed from document text (`5万円未満：部門長承認`). A revised 規程 silently invalidates them.	Rules are extracted from the document rather than typed into code, so re-extraction is the update path. Version the rule table against the document; alert on drift; require human sign-off on each extraction.
R6	**Provider availability, if a model is ever added.**	The first live API call fell through **two 503s** before succeeding, and the default model had been retired for new keys mid-project.	Already mitigated by design: the model is off the runtime path, and the tool works with none configured.
R7	**Free-tier LLM data is used for provider training.**	Vendor terms.	`src/llm.py` refuses any prompt containing raw-log markers, so only derived material can leave the machine. Enforced in code, not promised in a document.
R8	**Boundary precision is structurally limited.**	BF1@2s = 0.286; screen text is captured every ~7.7 s.	Do not build anything requiring sub-5-second boundary accuracy. If needed, raise capture frequency — a collection change, not an algorithm change.

7. How the seven days were spent

day	work	why
0	Data profiling; 790 MB of JSONL → an 11 MB index	Nothing else was affordable to iterate on until this existed
1	**Evaluation harness before the segmenter**; baselines	The brief leaves "good enough" to my judgment, and that judgment is impossible without a scorer. Building it first also killed the gap-based approach in an hour instead of a day
2	Case-ID recovery	The finding that reframed Step 1
3	v1 segmenter; LLM layer	v1 reached BF1@5s 0.450
4	Signature labelling; **dataset B transfer failed**; two metric audits	The failure was the most valuable day — see below
5	v3 terminator-driven segmenter; overfitting audit; Step 2 analysis	BF1@5s 0.450 → 0.722
6	Step 3 engine, three definitions, mock portal	94% automated, zero failures
7	This report; verification of the Japanese readings	

The allocation I would defend: roughly a third of the time went to measurement rather than building — the harness, three audits, the held-out protocol, the transfer checks. That is a high proportion, and it paid for itself three times over. The metric defect (§1) would have made every subsequent number wrong. The dataset B transfer failure was invisible on every internal statistic — coverage, label count and durations all looked plausible — and was caught only by a signal deliberately withheld from the method. The overfitting audit found a leaked protocol.

The day I would change: Day 3's LLM layer was built because it seemed required, a full three days before anything needed it, and the eventual answer was that it should not be in the runtime path at all. That time would have been better spent on Step 2.

---

8. Honest limitations

- **Dataset B has no ground truth**, so its accuracy is not measured, only inferred from dataset A performance and held-out proxy checks. The strongest evidence is that 149 completion memos never read by the pipeline land at median relative position **0.78** within predicted segments, against 0.35 for the version that was wrong.
- **Variant detection is weak on dataset B.** The 種別 column carries 定常/調整, but only 72 of 668 segments can currently be tied to one, and their durations barely differ. Reported as weakly evidenced rather than as a finding.
- **10% of dataset A screenshots are missing** from the distribution provided. Not pursued: dataset B is complete and screen text is available as text.
- **"36% of work at 94% coverage" describes 175 observed minutes on one day with four operators.** It is a measurement of this sample, not an estimate of the client's operation.

What I would do next, in order

1. Run the engine against a staging instance of the real portal (R2).
2. Check whether an API exists behind the portal. If it does, most of this tool becomes unnecessary, and that is a good outcome.
3. Instrument L3 coverage per machine as a health metric (R1).
4. Extend to `leave-applications` — 25.5% of work, but needs the procedure-following problem solved first.